

FASTNav: Fine-Tuned Adaptive Small-Language-Models Trained for Multi-Point Robot Navigation

Yuxuan Chen[✉], Yixin Han[✉], and Xiao Li[✉]

Abstract—With the rapid development of large language models (LLM), robots are starting to enjoy the benefits of new interaction methods that large language models bring. Because edge computing fulfills the needs for rapid response, privacy, and network autonomy, we believe it facilitates the extensive deployment of large models for robot navigation across various industries. To enable local deployment of language models on edge devices, we adopt some model boosting methods. In this paper, we propose *FASTNav* - a method for boosting lightweight LLMs, also known as small language models (SLMs), for robot navigation. The proposed method contains three modules: fine-tuning, teacher-student iteration, and language-based multi-point robot navigation. We train and evaluate models with FASTNav in both simulation and real robots, proving that we can deploy them with low cost, high accuracy and low response time. Compared to other model compression methods, FASTNav shows potential in the local deployment of language models and tends to be a promising solution for language-guided robot navigation on edge devices.

Index Terms—Indoor navigation, large language models, path planning.

I. INTRODUCTION

THE adoption of robots has significantly expanded across various domains, including medical, household, and industrial sectors, driven by the rapid advancements in robotics technology. However, traditional robot control methods are limited to simple tasks and are inadequate for handling complex environments and tasks described in natural language. Large language models (LLMs) such as ChatGPT and Llama have demonstrated exceptional natural language processing and logical reasoning abilities, highlighting their potential for enhancing robot navigation. Current approaches predominantly depend on API calls to leverage LLMs, which pose challenges in terms of

privacy protection and real-time responsiveness, thereby limiting their practical application. In this work, we try to address the following question: “*can we develop a robot navigation system based on language models that (1) is lightweight and deployable locally, and (2) is computationally inexpensive while exhibiting high performance for complicated navigation tasks?*”

Recently, with the integration of robot control and LLMs, many innovative ideas have emerged. Some zero-shot or few-shot methods that use pre-trained models for navigation without the need for fine-tuning or annotating data [1], [2] have emerged. Additionally, there are methods that use spatial map representations to directly combine pre-trained visual-linguistic features with 3D reconstructions of the physical world [3]. While these methods excel in perceiving the real world, they exhibit inefficiencies in the LLM component, as indicated by the “fast-forward” signs frequently seen in demonstration videos. By reducing the size and complexity of large models, model compression methods can significantly decrease computational load and inference time, offering a promising solution to address those inefficiencies. Additionally, these techniques enable the deployment of lightweight models on edge devices, ensuring more efficient resource usage without sacrificing too much performance. However, when compressing, most of these methods require long running times and high computing power and do not perform well for specific tasks.

To solve all these problems above, we propose a technology of model boosting. We draw inspiration from model compression, leveraging knowledge transfer from LLMs to small language models (SLMs) through prompting and feedback. This way enhances the capabilities of the smaller model while preserving its lightweight nature. **Our approach involves first fine-tuning the SLMs using domain-specific datasets and then applying a teacher-student iteration module to further improve their performance.** This enables SLMs to achieve performance levels close to those of much larger models in robot navigation tasks, while also offering high feasibility for practical deployment. The illustration of our system is shown in Fig. 1.

To address the challenge of deploying language models locally on robots, we select smaller models and fine-tune them with LoRA [4]. This enhances the models’ inference speed and accuracy without requiring extensive prompt engineering. Additionally, we design a teacher-student iteration module to enable the models to continuously learn from navigation tasks. To summarize our contributions, we

Received 12 June 2024; accepted 13 November 2024. Date of publication 25 November 2024; date of current version 5 December 2024. This article was recommended for publication by Associate Editor S. Thakar and Editor A. Banerjee upon evaluation of the reviewers’ comments. This work was supported by NSFC under Grant 52405029. (Yuxuan Chen and Yixin Han contributed equally to this work.) (Corresponding author: Xiao Li.)

Yuxuan Chen and Yixin Han are with the School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China (e-mail: chen_yuxuan@sjtu.edu.cn; hanyixin@sjtu.edu.cn).

Xiao Li is with the School of Mechanical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China, and also with the State Key Laboratory of Mechanical System and Vibration, Shanghai 200240, China (e-mail: sjtu_lixiao@sjtu.edu.cn).

This letter has supplementary downloadable material available at <https://doi.org/10.1109/LRA.2024.3506280>, provided by the authors.

Digital Object Identifier 10.1109/LRA.2024.3506280

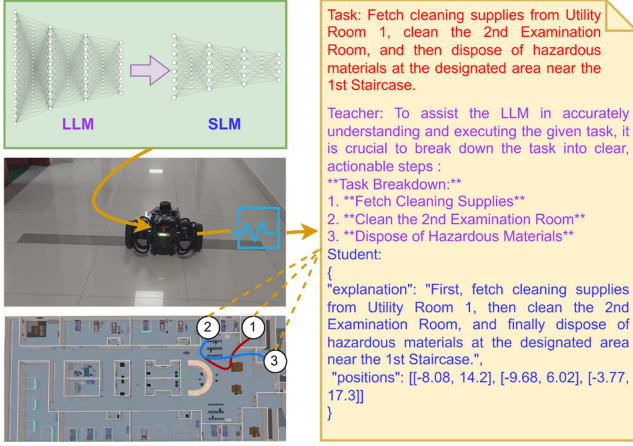


Fig. 1. **FASTNav** is a method for boosting SLMs for robot navigation. We improve the performance of SLMs for multi-point robot navigation by fine-tuning and teacher-student iteration.

- explored an innovative method of model boosting that boosts the performance of SLMs to levels comparable to those of much larger language models for specific domain tasks;
- introduced FASTNav, a novel method utilizing fine-tuned adaptive small-language-models (SLMs) for multi-point robot navigation;
- evaluated FASTNav in both simulated and real environments, demonstrating significant improvements in success rates and implementation efficiency, leading to a network-independent, large model-based robot navigation approach that ensures user privacy and data security while providing a cost-effective, high-performance solution.

In this work, we focus on boosting SLMs to bring their performance in robot navigation closer to that of much larger models, enabling robots to interact with users accurately and promptly.

II. RELATED WORK

Using LLMs for robot navigation has become a significant research field in robotics control technology, significantly impacting autonomous driving technologies. Target navigation points can be determined based on users' instructions using vision [5], landmark [6], olfaction [7] and large language model technologies [8], [9]. In recent years, technologies combining LLMs with visual modules have been widely used in robot navigation [1], [10], showing excellent results. *However, constrained by the parameter sizes of small models, these methods do not show significant improvements on local small models.*

Applying LLMs to planning tasks represents a growing area of research aimed at enhancing the capabilities of autonomous agents. Some methods leverage LLMs to translate planning problems into Planning Domain Definition Language (PDDL) format, achieving strong performance on specific tasks [11], [12]. Some other approaches use Signal Temporal Logic (STL) as an intermediate representation, endowing LLMs with much

stronger reasoning and planning capabilities than natural language alone [13]. *Even so, these methods face challenges such as high computational costs, slow response times, and limited adaptability in complex environments.*

Model compression methods for LLMs are already numerous. Pruning reduces the size and computational cost of a neural network by removing less important weights or neurons while attempting to preserve the model's performance [14], [15]. Knowledge distillation transfers the knowledge from a large model (teacher model) to a smaller model (student model), enabling the smaller model to achieve comparable performance [16], [17]. Quantization compresses a model by reducing the precision of its weights and activations, typically by converting them from floating-point to lower-bit representations, thereby saving memory and computational resources [18], [19]. Low-rank factorization approximates the weight matrices of a model with lower-rank matrices, reducing the number of parameters and computational complexity without significantly degrading performance [20]. *However, most of these methods require long running times and high computing power requirements, and do not perform well for specific tasks.*

Navigation with SLMs can speed up operation and benefit privacy protection. Research on controlling robots with local SLMs is rare, and methods to improve the performance of local small models are scarce. To fill this gap, we start to attempt related research. We aim to enhance the performance of local models through various comprehensive methods, including fine-tuning language models using datasets, supporting them with prompt engineering, and exploring the limits of local SLMs' performance using the teacher-student iteration, to improve accuracy in multi-target navigation tasks.

III. BACKGROUND

A. Parameter-Efficient Fine-Tuning Based on LoRA

A key issue with SLMs is their limited logical reasoning and analytical capabilities. Additionally, their output format may not be robust enough for direct application to domain-specific tasks. Therefore, it is important to fine-tune SLMs to adapt them to the specific domain of robot navigation. Some researches [21], [22] have shown that with appropriate fine-tuning, SLMs can achieve performance close to that of LLMs.

The fine-tuning method we use is LoRA [4], Low-Rank Adaptation of LLMs, which is an innovative method for parameter-efficient fine-tuning.

LoRA represents parameter updates $\Delta\Phi$ with a much smaller set of parameters compared to the full set Θ . This is done by using a low-rank matrix to encode the weight update ΔW as:

$$W_0 + \Delta W = W_0 + BA \quad (1)$$

where W_0 is the pre-trained weight matrix, $B \in \mathbb{R}^{d \times r}$, and $A \in \mathbb{R}^{r \times k}$, with $r \ll \min(d, k)$. During training, W_0 is kept frozen, and only A and B are updated.

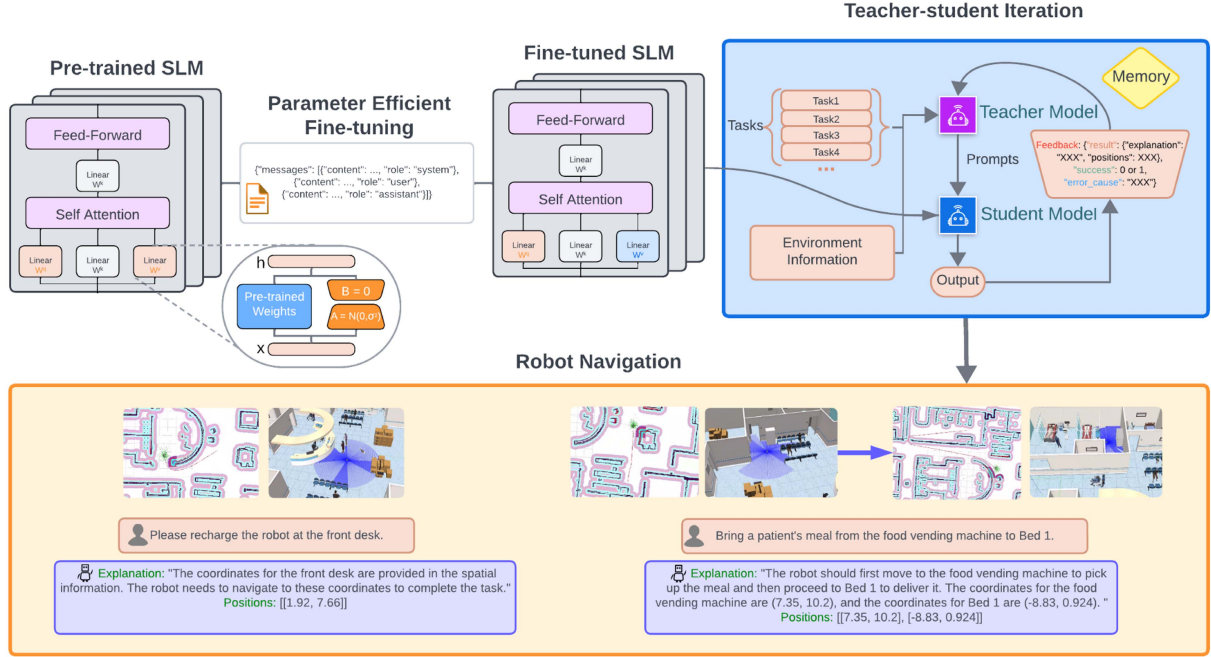


Fig. 2. Architecture of FASTNav. The architecture contains three components: (1) fine-tuning of SLMs based on PEFT: we use PEFT to give SLMs access to the necessary environment information and to constrain their output format; (2) teacher-student iteration: the fine-tuned SLMs cooperate with teacher models to solve navigation tasks, where the teacher serves as a prompt engineer and feedback receiver, transferring knowledge and helping SLMs correct mistakes and (3) robot navigation: the robot receives an ordered list of coordinate points from SLMs and moves with the navigation algorithms according to the list.

B. Language Navigation for Robots

We conceptualize the Language Navigation for Robot problem as follows: Given a natural language command W , comprised of a sequence of words $w_1, w_2, w_3, \dots, w_{n_w}$, the language model is tasked with making navigation decisions based on a pre-constructed map representation M . Upon receiving the command, the model leverages M to determine a series of target waypoints that define the navigation path.

The decision-making process can be formalized by the function f , which maps the language command and the map information to the sequence of waypoints: $f(W, M) = \{p_1, p_2, \dots, p_n\}$, where each p_i is a coordinate or identifier for a location on the map M .

The primary objective of the language model is to optimize the sequence of waypoints such that it accurately reflects the intent of the language directive while adhering to the constraints and affordances of the map M . Therefore, the most critical formula representing the decision-making function in isolation is:

$$f(W, M) = \{p_1, p_2, \dots, p_n\} \quad (2)$$

This function encapsulates the essence of this problem, translating a linguistic input into a navigational output within the context of a given spatial representation.

IV. PROBLEM DEFINITION AND APPROACH

Our goal is to fetch a lightweight SLM that performs well for robot navigation tasks by fine-tuning and teacher-student iteration. Formally, the input is a natural language task description L and map information M , and the output is a sequence of target coordinates $T = \{t_1, t_2, \dots, t_k\}$ arranged in order.

First, we fine-tune the SLM $f_{\theta_s}(L, M)$, where L is the task description and M is the map data. The goal here is to minimize the fine-tuning loss $\mathcal{L}_{FT}$, ensuring the model can accurately predict the sequence of target coordinates $T = \{t_1, t_2, \dots, t_k\}$ for navigation.

Once fine-tuning is complete, we improve the SLM by learning from a larger model $f_{\theta_l}(L, M)$. This process tries to reduce both the task-specific loss $\mathcal{L}_{FT}$ and the knowledge distillation loss $\mathcal{L}_{TS}$, allowing the small model to refine its predictions based on the larger model's output.

Our approach ensures the small model can effectively generate robot navigation paths from natural language descriptions and map data. It is assumed that most objects' positions on the map (including landmarks) remain stable, though it allows for the possibility of small, dynamic obstacles. By combining fine-tuning with knowledge from the larger model, we aim to enhance the small model's performance while maintaining its efficiency for real-world tasks.

V. FASTNAV

In this section, we present the general structure of our approach. FASTNav is divided into three main sections: (1) fine-tuning of small language models; (2) teacher-student iteration and (3) robot navigation controller. Fig. 2 provides a general overview of the structure.

A. Fine-Tuning of SLMs

Considering the shortcomings that SLMs have when dealing with specific tasks, a fine-tuning process is needed to adapt them to the specific domain of robot navigation. During the

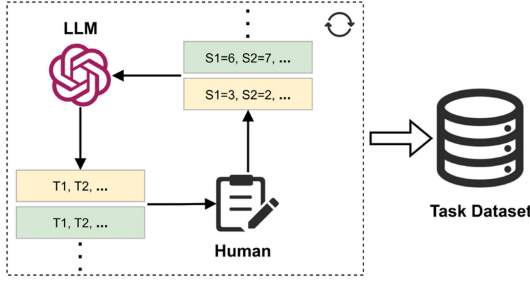


Fig. 3. Human-in-the-loop generation of the dataset. The human-in-the-loop generation is used to generate our dataset for fine-tuning, which is a valid method for obtaining datasets aligned with humans. The yellow box corresponds to the first cycle, and the green box corresponds to the second cycle. T_i means the i -th task while S_i means the score that T_i gets.

fine-tuning process, the expected output of language models is a JSON format context that includes *explanation* and *positions*. The *explanation* is a string that shows the model's analysis and reasoning about the task, making it easy to understand its thought process and to see why the task succeeds or fails. The *positions* is a list of pairs of numbers containing the x and y coordinates of the target points. We hope SLMs can output in such a format, utilizing the *positions* to navigate the robot and showing its thought process through the *explanation*.

Human-in-the-loop generation is used to generate our dataset for fine-tuning. We first write some tasks using examples from real life. The tasks and some information about the environment are composed to make a prompt. Then an LLM generates some tasks according to the prompt. A human evaluator joins the process and gives scores to the tasks. The tasks and the scores are brought back to the LLM, and it generates a set of new tasks with this feedback from humans. We continue this cycle as shown in Fig. 3 again and again until we get enough high-quality data. After several cycles, a high-quality dataset that meets human requirements is finally obtained.

Here, we choose LoRA for fine-tuning due to its efficient parameter use, low inference overhead, and broad applicability. LoRA strikes a balance between performance and ease of use. It also benefits from strong community support and seamless integration with libraries.

We use the dataset to fine-tune our SLMs so that they will output the correct JSON text as expected. Here we use Huggingface's PEFT library [23] to help us quickly implement this process. During the fine-tuning process, we adjust parameters such as learning rate and epochs to obtain the accuracy of the SLMs on the test set under different parameters for comparison. The details of fine-tuning are displayed in Section VI.

B. Teacher-Student Iteration

The teacher-student model is a paradigm that has been proven effective in the context of language model training and knowledge transfer [24]. Our method is rooted in the principles of multi-agent systems, where multiple agents collaborate and learn from each other to achieve common goals [25], [26].

Algorithm 1: Teacher-student Iteration.

```

1: Inputs: tasks  $T$ ; environment information  $E$ ; names of
   models  $N$ ; expected goals  $G$ 
2:  $M^{teacher}, M^{student} \leftarrow \text{Initialize}(N)$ 
3:  $iteration = 1$ 
4: while  $iteration < \max(iter)$  do
5:   for  $i=1 \dots n$  do
6:      $f = \text{ReadFeedback}(M^{teacher})$ 
7:      $p^t = \text{GeneratePrompt}(M^{student}, f, T_i)$ 
8:      $result = \text{Completion}(p^t, T_i, E)$ 
9:      $state = \text{Navigation}(result)$ 
10:    if  $result[positions] == G_i$  then
11:       $success = 1$ 
12:    else
13:       $success = 0$ 
14:    end if
15:     $\text{UpdateFeedback}(result, success)$ 
16:  end for
17:   $iteration = iteration + 1$ 
18: end while

```

This method involves a more knowledgeable teacher model guiding a less knowledgeable student model, with the goal of improving the student's performance through iterative learning and feedback. The overall framework of the method is illustrated in Fig. 2.

A strong language model, such as GPT-4, is designated as the teacher, while a weaker SLM acts as the student. In each iteration, the teacher serves as a prompt engineer, responsible for generating suitable prompts for the student based on the current task and map environment information. The teacher also receives feedback from the previous iteration's results and adjusts the prompts' content in real-time based on this feedback. This ensures that the student can make correct judgments and attempt to correct any incorrect judgments as much as possible.

The student is typically a fine-tuned SLM designed to receive prompts from the teacher and generate corresponding outputs based on the current task. During each iteration, the student's output is recorded and used to provide feedback to the teacher in the subsequent iteration.

Algorithm 1 describes the procedures of the teacher-student iteration.

C. Robot Navigation Controller

In our current setup, SLMs serve as task planning tools, taking map landmarks (coordinates and attributes) and task descriptions as input. They generate an ordered sequence of target coordinates, which low-level navigation algorithms use to move the robot. The inputs and outputs of the SLMs can be seen in Figs. 1 and 2. Thus, after gaining the list of goal points from the language model, a robot navigation controller is required to direct the robot toward the points and update the planned path in real-time.

Here we choose Navigation2 [27], a modular navigation framework. We extract the list of goal points from the SLM's

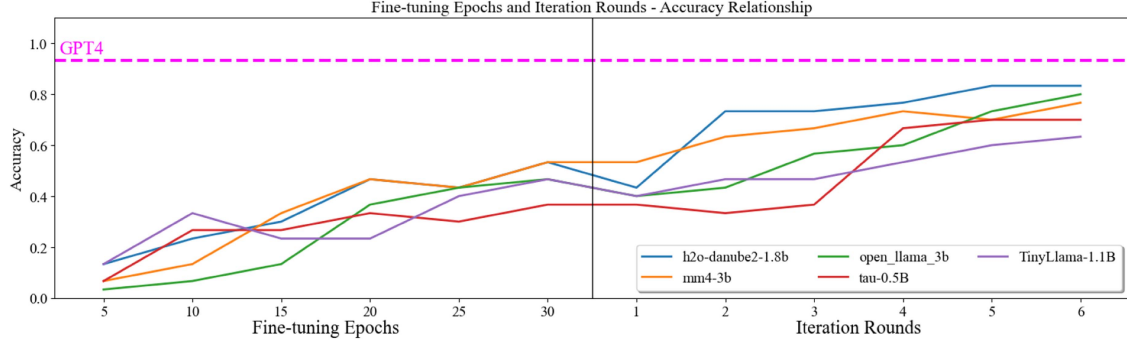


Fig. 4. The accuracy curve during fine-tuning and iteration. This graph shows the accuracy of five models and GPT4 on the test set during fine-tuning and iteration processes. It can be seen that all the models' accuracy rises gradually in both processes and finally gets close to that of GPT4. (The last fine-tuning epoch is immediately followed by the first round of iterations, which is why they continue on the same axis.).

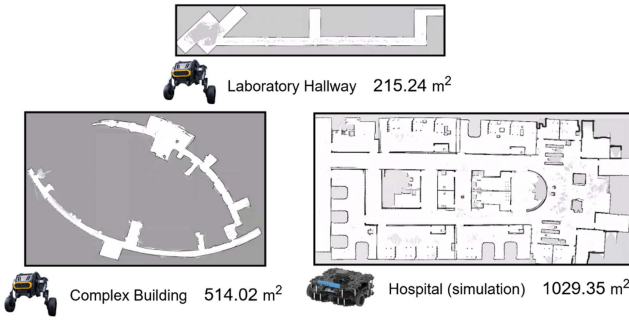


Fig. 5. The sizes of the environments used in our experiments. The lab hallway and complex building are used in the real-world experiments, while the hospital is used in the simulation experiment.

output and put it into Navigation2. The Nav2 controller then helps the robot navigate from the initial position to the expected points sequentially according to the list.

With all of these components, FASTNav is able to finish the entire navigation process. It receives a task command described in natural language, analyses the task logically, and navigates the robot to the right points sequentially.

VI. EXPERIMENTS AND RESULTS

A. Setup

Environment and Dataset: We use a virtual environment for simulation and two real environments for testing the robot. The simulation environment employs a hospital scenario [28], in which we design approximately 1400 tasks. These tasks are distributed with single goal-point tasks, two-goal-point tasks, three-goal-point tasks, and multiple (four or more) goal-point tasks in a ratio of 1:3:2:1. We use the data to fine-tune the model and additionally design about 100 tasks as a test set.

We use the corridor of our laboratory and a complex building as the real environments. Considering the limitations of real robot experiments, we design only about 100 tasks here for fine-tuning, using 10 tasks to test the robot and verify the feasibility of our approach to the robot. Fig. 5 shows the sizes of these different environments.

Implementation Details: An NVIDIA GeForce RTX 4090 is used for model fine-tuning and iteration. For simulation experiments, we select a TurtleBot3 Waffle. For robot experiments, we choose Direct Drive Tech's DIABLO, a direct-drive, wheel-footed robot. We deploy our system on the robot using an NVIDIA Jetson Orin NX 16GB as an edge device.

Methods of Evaluation: We evaluate our method and comparison cases in terms of *accuracy*, *efficiency* and *robustness*. We choose 4 metrics to evaluate our method: *Success Rate (SR)*, *Navigation Error (NE)*, *Average Time (AT)*, *Moving Time Ratio (MTR)*.

$$SR = \frac{1}{n} \sum_{i=1}^n S_i \quad (3)$$

$$NE = \frac{1}{n} \sum_{i=1}^n d(\hat{g}_i, g_i) \quad (4)$$

$$AT = \frac{1}{n} \sum_{i=1}^n T_i \quad (5)$$

$$MTR = \frac{1}{n} \sum_{i=1}^n \frac{T_{mi}}{T_i} \quad (6)$$

In (3), S_i is a flag for each task's success or failure. S_i is 1 if the task succeeds and 0 if it fails. In (4), $\hat{g}_i$ is the estimated value of the i -th goal point, and g_i is the ground truth of the i -th goal point. $d(\hat{g}_i, g_i)$ means the distance between $\hat{g}_i$ and g_i . In (5), T_i is the time spent finishing each task. As for (6), T_{mi} is the time spent by the robot to move while executing the i -th task. We denote MTR by the ratio of T_{mi} to T_i .

Comparison cases: Five SLMs and other compression methods are used for comparison. Due to hardware performance and energy consumption constraints, running LLMs with over 10B parameters on edge devices is difficult, so we select SLMs with fewer than 3B parameters. We choose 5 SLMs that perform well on public benchmarks according to the Open LLM Leaderboard on HuggingFace, as shown in Table I. Some models obtained through quantization and knowledge distillation were also selected. Quantization method GTPQ [18] is used on Llama3-8B [29] and Mistral-7B [30], while an int4 version of

TABLE I
ATTRIBUTES OF DIFFERENT SLMS

Model	Params (B)	Performance (%)	Memory (GB)	Tokens/s
tau-0.5B	0.5	36.68	4.2	16.90
TinyLlama-1.1B	1.1	36.42	6.3	12.51
danube-1.8	1.83	48.72	6.7	13.80
openllama-3B	3	38.26	10.7	8.42
mm4-3B	3	53.22	11.3	7.19

Qwen-7B-Chat [31] and MiniMA-3B [32] that is distilled from LLaMA2-7B are also selected.

To compare FASTNav with other LLM-based planning and navigation methods, we design three comparison setups and compare them with FASTNav in our simulation environment. LLM+P [11] integrates LLMs with classical planners by converting natural language tasks into Planning Domain Definition Language (PDDL) for task-solving, then translating the solution back into natural language. AutoTAMP [13] uses LLMs to translate tasks into signal temporal logic (STL), which a task and motion planner (TAMP) uses to generate motion plans. LLM-As-Task Planner [13] lets LLMs directly generate subtasks from instructions, with motion planning handled by a separate algorithm.

B. Results and Discussion

We start by testing the inference speed and computational resource usage of several lightweight language models on the NVIDIA Jetson Orin NX to evaluate their performance on an edge device. Next, we modify the SLMS with FASTNav. To verify the effectiveness of our method, the SLMS with FASTNav are compared with some quantified and distilled models with similar memory usage. We also test FASTNav and 3 different LLM-based planning and navigation methods in the simulation environment. Through ablation studies, we determine the specific roles of fine-tuning and iteration in FASTNav. At last, FASTNav is used on a real robot to make an instant and accurate local deployment.

The inference speed of SLMS tends to increase as the number of model parameters decreases, while memory usage increases proportional to the number of parameters: The models listed in Table I are deployed on the NVIDIA Jetson Orin NX to explore how their inference speed and computational resource requirements vary with different parameter counts. We measure the inference speed of the models in terms of tokens generated per second and the computational resources consumed in terms of memory used at runtime. The results in Table I show that inference speeds up for different models as the number of parameters decreases. And models' memory requirements increase with the number of parameters. Overall, the results demonstrate the feasibility of using SLMS for lightweight inference at the edge. We look forward to following up with a model that is both fast and accurate enough to be suitable for local deployment on robots.

SLMS, after FASTNav's fine-tuning and iteration, achieve a 4-fold increase in accuracy while maintaining their lightweight

TABLE II
COMPARISON OF MODEL PERFORMANCE BETWEEN FASTNAV AND OTHER COMPRESSION METHODS

Models	Memory (GB)	NE (m)	SR (%)	MTR
Llama3-8B-GPTQ	8	0.25	36.67	0.941
Mistral-7B-GPTQ	6	0.32	23.33	0.943
Qwen-7B-Chat-Int4	9	0.21	13.33	0.945
distilled-MiniMA-3B	8	0.16	6.67	0.966
openllama-3B with FASTNav	8	0.2	70	0.991
tau-0.5B with FASTNav	2	0.19	63.33	0.993

TABLE III
SUCCESS RATE AND INFERENCE TIME COMPARISON

Method	Success Rate (%)	Inference Time (s)
LLM+P (GPT4)	76.67	15.38
LLM+P (openllama-3B)	0	—
AutoTAMP (GPT4)	53.33	224.66
AutoTAMP (openllama-3B)	0	—
LLM-As-Task Planner (GPT4)	46.67	34.24
LLM-As-Task Planner (openllama-3B)	0	—
FASTNav (openllama-3B)	70	2.87

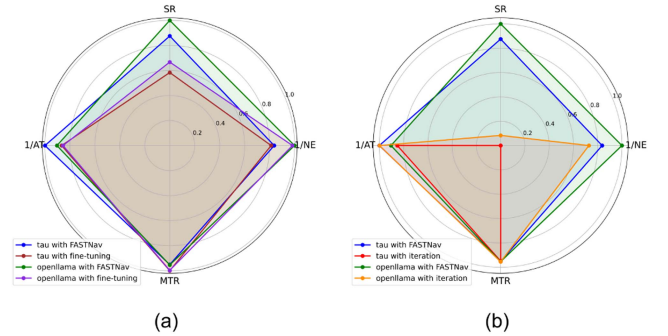


Fig. 6. Ablation experiments. (a) FASTNav method and fine-tuning-only method. Through FASTNav, models apparently have better SR than those only fine-tuned. (b) FASTNav method and iteration-only method. SLMS without fine-tuning perform terribly compared to SLMS with FASTNav.

nature: Through fine-tuning, the model's output format is standardized, making it easier for the robot to read coordinates. Additionally, the model's understanding of maps deepens, demonstrating improved responsiveness to human commands that do not contain specific location information. Multiple rounds of fine-tuning on the training dataset encode the relevant positional information into the model's parameters. We record the model's performance on the test set after each epoch, using accuracy as the metric. To further improve the performance of SLMS, we test our method in the simulation environment. Five selected models iterate through the teacher-student iteration to continuously optimize their performance. We recorded the model's accuracy after each iteration and plotted the curve showing the accuracy changes with the iteration number. Fig. 4 shows that, during fine-tuning, as the number of training epochs increases, the model's accuracy gradually improves and stabilizes around a certain value. The final results far exceed those of the original models, approaching the level of GPT-4, which aligns with our expected experimental results. This conclusion applies generally to the five models we selected, indicating a universal pattern. Therefore, we consider FASTNav to be effective for multi-point navigation tasks.

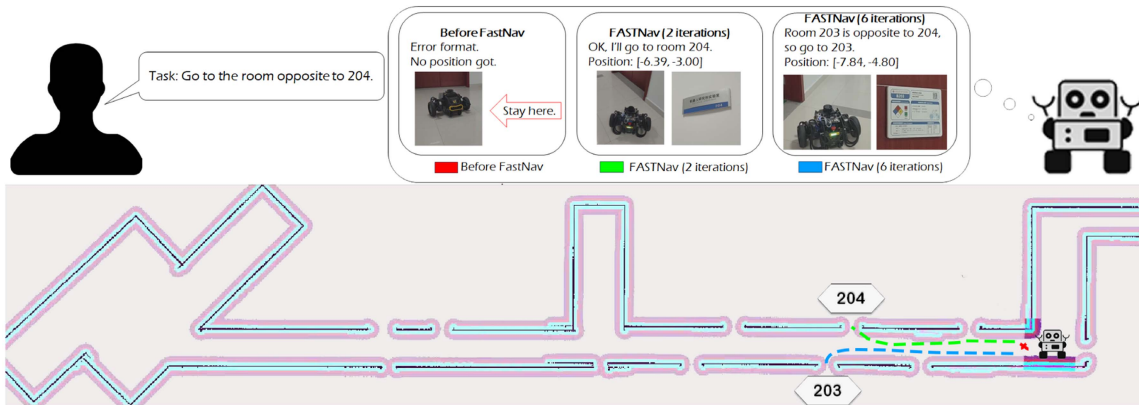


Fig. 7. Performance of the language model on one task under different conditions. The image compares the model's performance on one task on a real robot in 3 different configurations: without FASTNav, with FASTNav (2 iterations) and with FASTNav (6 iterations). The robot's capabilities significantly improved before and after FASTNav.

Compared to the selected LLM compression methods, FASTNav demonstrates an improvement of at least 30% in success rate and maintains a quicker response speed for navigation: Table II shows the memory usage of some comparisons and our chosen SLMs. Most models here cost 7–9 GB, which means the difficulties of deploying them locally are about the same. The SLMs with FASTNav here have the memory during the iterations, and the teacher models are removed so that they can respond fast. It can be seen that FASTNav with similar memory usage is remarkable, having better performance than comparisons. Regarding NE and SR, we can summarize that SLMs with FASTNav are more suitable for specific tasks. This is the result of fine-tuning and iteration. For AT and MTR, it is evident that our method is lighter and faster, resulting in more time spent on robots' movement instead of language models' inference. It may be because after fine-tuning, the models have learned the map information in the environment well, so we do not include this part in the prompt during querying. This allows the models' inputs to be drastically scaled down, with a corresponding reduction in the inference time. Especially for tau-0.5b with FASTNav, we find it has a fascinating performance that is close to that of much larger models, even with only 0.5B parameters. This phenomenon shows that FASTNav is able to fully tap the potential of SLMs, making it possible to deploy language models with low cost but high performance.

FASTNav demonstrates clear advantages in both performance and inference speed compared to other LLM-based planning and navigation methods: Based on the results in Table III, FASTNav shows clear advantages in robot navigation tasks compared to other methods. Unadjusted SLMs like openllama-3B struggle with tasks that require converting natural language into structured formats such as PDDL or STL, highlighting the importance of fine-tuning to constrain output formats. FASTNav, with fine-tuning and teacher-student iteration, achieves a 70% success rate, outperforming most methods using GPT-4, except for LLM+P, which has a slightly higher success rate of 76.67%. Additionally, FASTNav's inference time of 2.87 seconds is significantly lower than the other methods, which require complex reasoning and multiple rounds of prompts. This

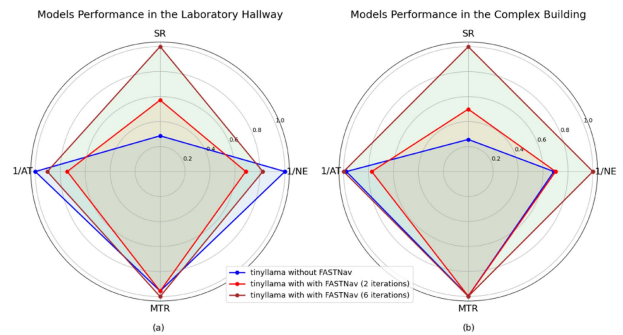


Fig. 8. Experiments on a real robot. After being processed by FASTNav, TinyLlama-1.1B shows higher SR and better performance on a real robot than before adjustments. Increasing the number of iterations also enhances the model's capabilities. We conducted experiments in two real-world environments of different sizes, demonstrating the method's real-world viability.

balance of efficiency and accuracy, along with its competitive performance, indicates that FASTNav is well-suited for real-world applications, with strong potential for future scalability and broader applicability in navigation tasks.

Through ablation, we find that fine-tuning can effectively constrain the output format and enhance task success rate, while teacher-student iteration is responsible for further improving the performance of fine-tuned models (reaching near teacher performance): To verify every module's role in our method, we design ablation experiments. For two chosen models, openllama and tau, we test their performance in the simulation environment in the case of with FASTNav, with iteration only, and with fine-tuning only. Results in Fig. 6(a) show that fine-tuning is the key to using SLMs on specific domains, which greatly constrain the output format of SLMs. Without fine-tuning, it is really hard for SLMs to give outputs truly suitable for our tasks. According to Fig. 6(b), on the basis of fine-tuning, the teacher-student iteration can further improve their abilities, leading to a 30%–40% SR increment. If we were to compare the performance of the two individually, we found that teacher-student iteration alone is inferior to fine-tuning alone. This is likely because fine-tuning reinforces map information in the SLM and constrains the output

format, which is crucial for our later use. In contrast, iteration alone is less effective due to the SLM's limited generalization capabilities. Combining these two modules, FASTNav is proven to go beyond fine-tuning-only or iteration-only methods.

FASTNav is an effective method for enhancing SLM performance, creating high-performing and edge-deployable language-guided robot navigation models: We deploy FASTNav on DIABLO, conducting experiments in a laboratory hallway and a complex building, and the results are shown in Figs. 7 and 8. We chose the TinyLlama-1.1B model for experiments on the robot. As shown in Fig. 8, the unadjusted TinyLlama language model has a low success rate and significant navigation errors. After 2 iterations and 6 iterations, the model's accuracy gradually increases, reaching a high level. The navigation errors gradually decrease, and its usability in real environments steadily improves. Based on a series of data, the language model fully adjusted by FASTNav also demonstrates superior performance in real environments.

C. Limitations

- FASTNav currently focuses on language models, missing other multimodal information like vision.
- FASTNav needs a large amount of data for fine-tuning, which is not always easy to get.
- FASTNav is primarily task-oriented, requiring specific fine-tuning and iteration for each different task environment.

In the future, we aim to explore the lightweight compression of multimodal language models that can process both visual and textual information, allowing our method to address more complex tasks by leveraging richer environmental data. Additionally, we plan to refine the structure of our approach to enhance its few-shot learning capability, reducing its reliance on extensive fine-tuning. This will improve its adaptability to diverse environments and enable it to perform well even with limited task-specific training data.

VII. CONCLUSION

In this work, we propose FASTNav, which represents fine-tuned adaptive small language models trained for multi-point robot navigation. This method includes three modules: fine-tuning, teacher-student iteration, and robot navigation controller. It is shown that SLMs can remain lightweight and have great performance close to much larger models in specific domains with FASTNav. We believe that FASTNav can greatly release the potential of SLMs and drive the widespread application of LLM technologies at the edge end.

REFERENCES

- [1] D. Shah, B. Osinski, B. Ichter, and S. Levine, "LM-Nav: Robotic navigation with large pre-trained models of language, vision, and action," in *Proc. Conf. Robot Learn.*, 2022, pp. 492–504.
- [2] M. Ahn et al., "Do as I can, not as I say: Grounding language in robotic affordances," in *Proc. Conf. Robot Learn.*, 2022, pp. 287–318.
- [3] C. Huang, O. Mees, A. Zeng, and W. Burgard, "Visual language maps for robot navigation," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 10608–10615.
- [4] E. J. Hu et al., "LoRA: Low-rank adaptation of large language models," in *Proc. Int. Conf. Learn. Representations*, 2022.
- [5] M. Adamkiewicz et al., "Vision-only robot navigation in a neural radiance world," *IEEE Robot. Automat. Lett.*, vol. 7, no. 2, pp. 4606–4613, Apr. 2022.
- [6] S. K. Harootyan, A. D. Ekstrom, and R. C. Wilson, "Combination and competition between path integration and landmark navigation in the estimation of heading direction," *PLoS Comput. Biol.*, vol. 18 2022, Art. no. e1009222.
- [7] L. Marques et al., "Olfaction-based mobile robot navigation," *Thin Solid Films*, vol. 418, no. 1, pp. 51–58, 2002.
- [8] D. Shah, M. R. Equi, B. Osinski, F. Xia, B. Ichter, and S. Levine, "Navigation with large language models: Semantic guesswork as a heuristic for planning," in *Proc. 7th Conf. Robot Learn.*, 2023, pp. 2683–2699.
- [9] W. Wang, L. Mao, R. Wang, and B.-C. Min, "SRLM: Human-in-loop interactive social robot navigation with large language model and deep reinforcement learning," 2024, *arXiv:2403.15648*.
- [10] W. Hao, C. Li, X. Li, L. Carin, and J. Gao, "Towards learning a generic agent for vision-and-language navigation via pre-training," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, 2020, pp. 13134–13143.
- [11] B. Liu et al., "LLM P: Empowering large language models with optimal planning proficiency," 2023, *arXiv:2304.11477*.
- [12] G. Dagan, F. Keller, and A. Lascarides, "Dynamic planning with a LLM," 2023, *arXiv:2308.06391*.
- [13] Y. Chen, J. Arkin, Y. Zhang, N. A. Roy, and C. Fan, "AutoTAMP: Autoregressive task and motion planning with LLMs as translators and checkers," in *Proc. IEEE Int. Conf. Robot. Automat.*, 2023, pp. 6695–6702.
- [14] M. Zhang et al., "LoRAPrune: Structured pruning meets low-rank parameter-efficient fine-tuning," in *Proc. Annu Meeting Assoc. Comput. Linguistics*, 2023.
- [15] X. Ma, G. Fang, and X. Wang, "LLM-pruner: On the structural pruning of large language models," in *Proc. Adv. Neural Inf. Process. Syst.*, 2023, pp. 21702–21720.
- [16] Y. Gu, L. Dong, F. Wei, and M. Huang, "Knowledge distillation of large language models," in *Proc. Int. Conf. Learn. Representations*, Jun. 2023.
- [17] Y. Huang, Y. Chen, Z. Yu, and K. McKeown, "In-context learning distillation: Transferring few-shot learning ability of pre-trained language models," 2022, *arXiv:2212.10670*.
- [18] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate quantization for generative pre-trained transformers," in *Proc. Int. Conf. Learn. Representations*, 2023.
- [19] Z. Liu et al., "LLM-QAT: Data-free quantization aware training for large language models," 2023, *arXiv:2305.17888*.
- [20] X. Wu, Z. Yao, and Y. He, "ZeroQuant-FP: A leap forward in LLMs post-training w4a8 quantization using floating-point formats," 2023, *arXiv:2307.09782*.
- [21] A. Mitra et al., "Orca 2: Teaching small language models how to reason," 2023, *arXiv:2311.11045*.
- [22] G. Juneja, S. Dutta, S. Chakrabarti, S. Manchanda, and T. Chakraborty, "Small language models fine-tuned to coordinate larger language models improve complex reasoning," in *Proc. Conf. Empirical Methods Natural Lang. Process.*, 2023, pp. 3675–3691.
- [23] S. Mangrulkar, S. Gugger, L. Debut, Y. Belkada, S. Paul, and B. Bossan, "PEFT: State-of-the-art parameter-efficient fine-tuning methods," 2022. [Online]. Available: <https://github.com/huggingface/peft>
- [24] D. Chen et al., "Grimoire is all you need for enhancing large language models," 2024, *arXiv:2401.03385*.
- [25] S. Hong et al., "MetaGPT: Meta programming for multi-agent collaborative framework," in *Proc. 12th Int. Conf. Learn. Representations*, 2024.
- [26] Q. Wu et al., "AutoGen: Enabling next-gen LLM applications via multi-agent conversation," in *Proc. Int. Conf. Learn. Representations Workshop Large Lang. Model*, 2024.
- [27] S. Macenski, F. Martín, R. White, and J. G. Clavero, "The marathon 2: A navigation system," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2020, pp. 2718–2725.
- [28] A. RoboMaker, "AWS-robomaker-hospital-world," 2000. [Online]. Available: <https://github.com/aws-robotics/aws-robomaker-hospital-world>
- [29] A. Dubey et al., "The llama 3 herd of models," 2024, *arXiv:2407.21783*.
- [30] A. Q. Jiang et al., "Mistral 7b," 2023, *arXiv:2310.06825*.
- [31] J. Bai et al., "Qwen technical report," 2023, *arXiv:2309.16609*.
- [32] C. Zhang, D. Song, Z. Ye, and Y. Gao, "Towards the law of capacity gap in distilling language models," 2023, *arXiv:2311.07052*.